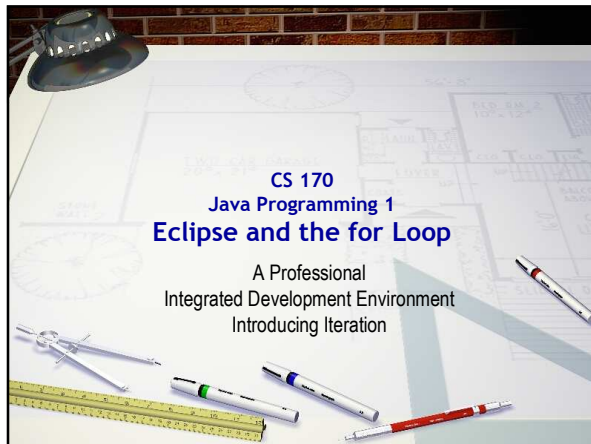
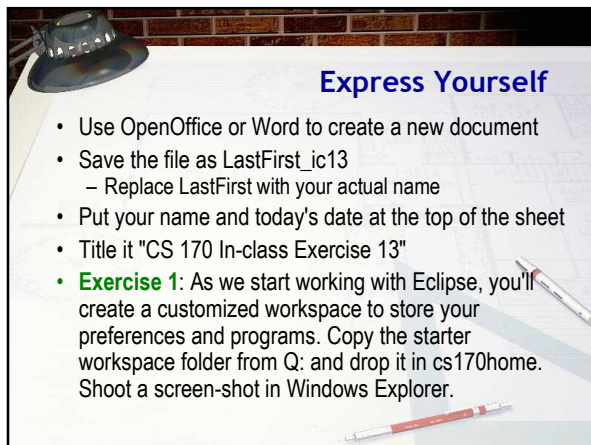
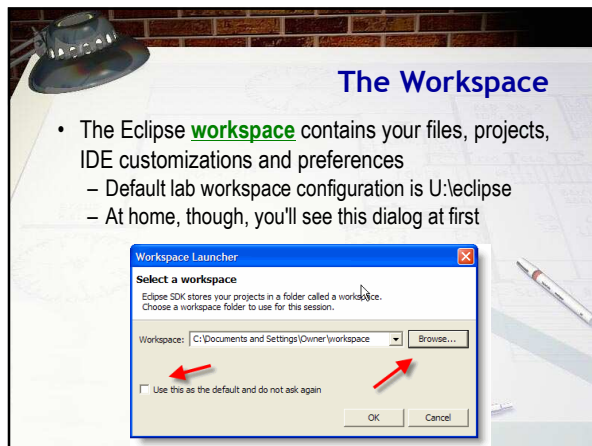


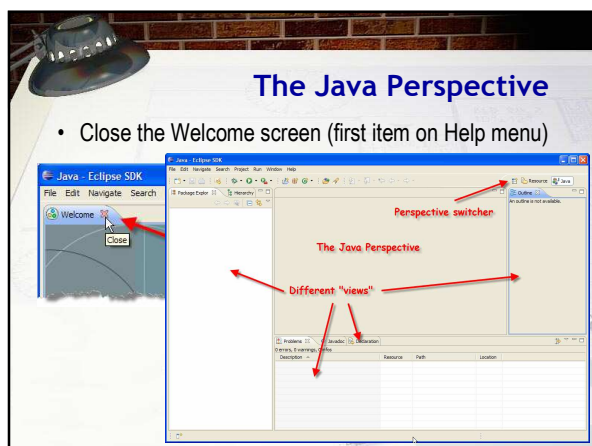






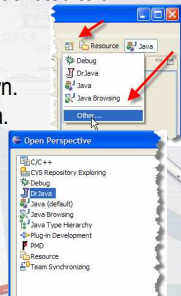






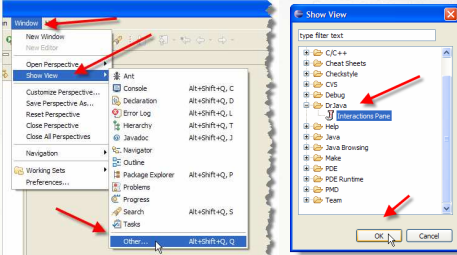
Perspectives

- Combination of windows and layout devoted to a specific task is called a **perspective**
- Exercise 3:** click Open Perspective button, choose Other from drop-down. Choose Resource and then Dr. Java. Shoot a screen-shot of each. Switch back to Java perspective.



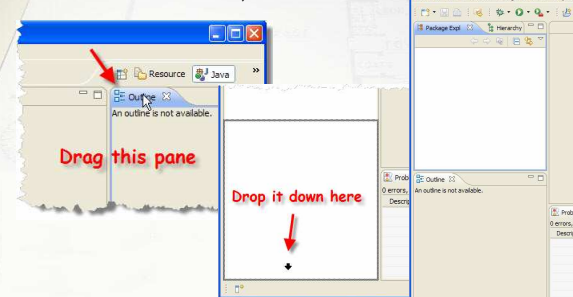
Views

- Individual windows in a perspective are called **views**. You can add or remove a view from any perspective.



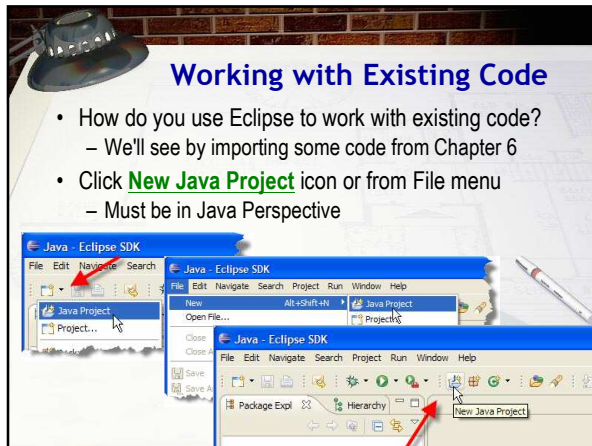
Arranging Views

- You can also move, float and dock individual views.



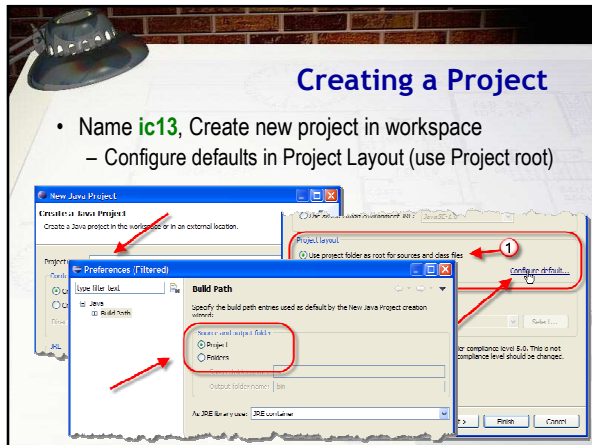
Working with Existing Code

- How do you use Eclipse to work with existing code?
 - We'll see by importing some code from Chapter 6
- Click **New Java Project** icon or from File menu
 - Must be in Java Perspective



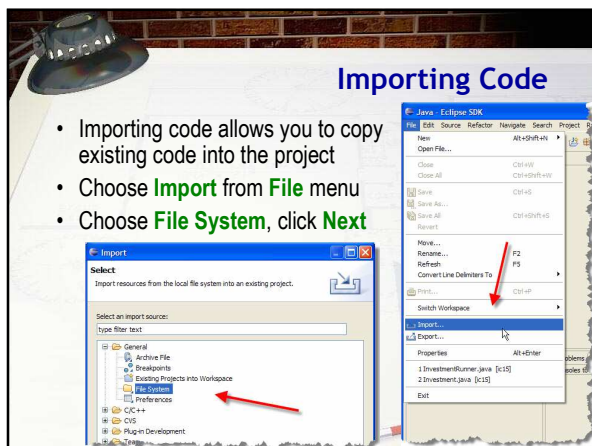
Creating a Project

- Name **ic13**, Create new project in workspace
 - Configure defaults in Project Layout (use Project root)



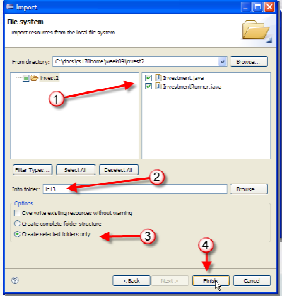
Importing Code

- Importing code allows you to copy existing code into the project
- Choose **Import** from **File** menu
- Choose **File System**, click **Next**



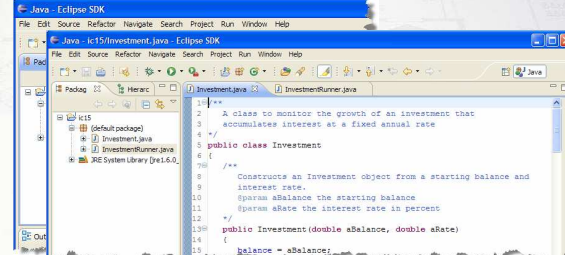
Importing Files

- Locate Q:\faculty5\sgilbert\cs170\code\week09
 - Select invest2 folder
 - Click **OK**
- Select 2 files
- Click **Finish**
- **Exercise 4:** snap a picture of window after importing files



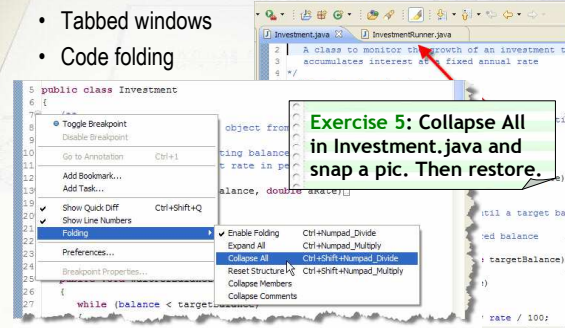
Package Explorer and Editor

- Files appear in Package Explorer window
 - Open default package and double-click source files

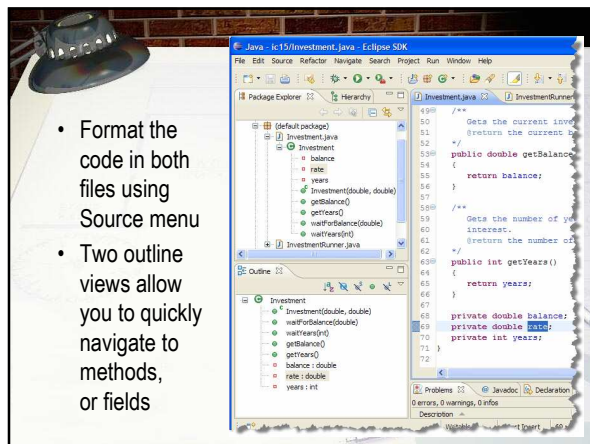


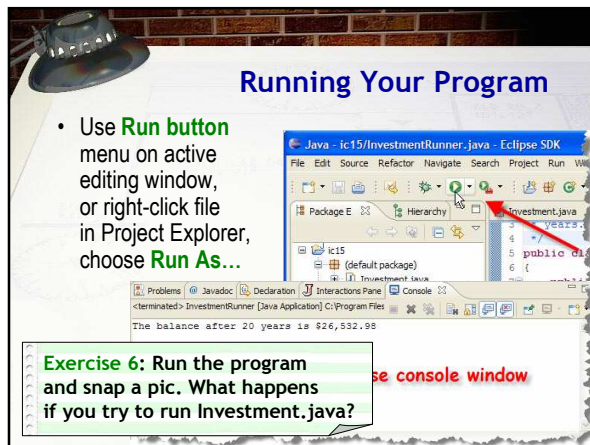
Editor Features

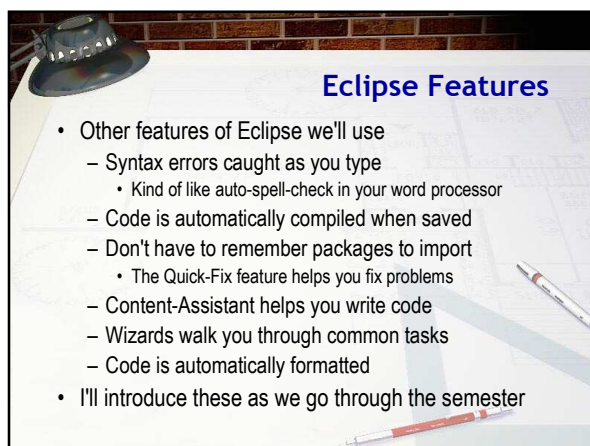
- Tabbed windows
- Code folding



Exercise 5: Collapse All in Investment.java and snap a pic. Then restore.







Investment.java

- Calculates investment value over n years (or number of years to reach a particular value).
 - Can't just multiply (or divide) by percentage
 - Invest \$ 1,000 @ 10% APR, compounded annually
 - First year you earn \$100 (10% of \$1,000)
 - Next year you earn \$110 (10% of \$1,100)
- Investment class
 - Fields: **balance**, **rate**, **years**
 - Constructor: initializes all fields
 - Accessors: **getBalance()** and **getYears()**

InvestmentRunner

- Create an investment, wait n years, get the balance

```
1 public class InvestmentRunner
2 {
3     public static void main(String[] args)
4     {
5         final double INITIAL_BALANCE = 10000;
6         final double RATE = 5;
7         final int YEARS = 20;
8
9         Investment invest = new Investment(INITIAL_BALANCE, RATE);
10        invest.waitYears(YEARS);
11        double balance = invest.getBalance();
12
13        System.out.printf("The balance after %d years is %,.2f\n", YEARS,
14                           balance);
15    }
16 }
```

Diagram illustrating the flow of execution in the `main` method of `InvestmentRunner`:

1. Create an `Investment` object: `Investment invest = new Investment(INITIAL_BALANCE, RATE);`
2. Call `waitYears`: `invest.waitYears(YEARS);`
3. Call `getBalance`: `double balance = invest.getBalance();`

The Future Value

- Using a **for** loop to calculate the value after n years

```
35 /**
36  * Keeps accumulating interest for a given number of years.
37  * @param n the number of years
38  */
39 public void waitYears(int n)
40 {
41     for (int i = 1; i <= n; i++)
42     {
43         double interest = balance * rate / 100;
44         balance = balance + interest;
45     }
46     years = years + n;
47 }
48 /**
```



What Is Iteration?

- A CS term that means **repeating a set of actions**
 - Also called **repetition** or **looping**
 - Statements used in iteration are called **loops**
- Loops are **flow-of-control** statements **similar** to **if**
 - Like the **if** statement, a loop evaluates a condition
 - If the condition is true, a group of actions are performed
 - If the condition is false, the actions are skipped

```
graph TD
    Start(( )) --> Decision{TIRED? REST AREA 3/4 MILE FREE COFFEE}
    Decision -- Yes --> JavaMatic[JavaMatic]
    JavaMatic --> Decision
    Decision -- No --> Exit(( ))
```

Iteration and Selection

- Iteration is **not the same** as selection, however
- A loop **returns to its test** after performing its actions
- If the test is still true, then the actions are **performed again**
- A selection statement only performs its actions **once**
- You cannot replace a loop with a selection statement

```
graph TD
    Start(( )) --> Decision{TIRED? REST AREA 3/4 MILE FREE COFFEE}
    Decision -- Yes --> JavaMatic[JavaMatic]
    JavaMatic --> Decision
    Decision -- No --> Exit(( ))
```

Java and Iteration

- Java has three loops: **while**, **do-while**, and **for**
 - Each designed to work best in a particular situation
- One difference is **where** the test is made
 - Test at the bottom
 - Test at the top

```
{  
    // Loop Body  
}
```

Loop Test

Loop Test

```
{  
    // Loop Body  
}
```

Loop Classification

- Where tested is **not the best** method of classification
 - Doesn't provide any empirical guidance
 - Better to ask **how each loop is controlled**
- Two basic methods of loop control
 - Simplest method relies on a counter
 - Called **counting** or **counted** loops
 - "Repeat these actions 10 times"
 - More complex loops test the occurrence of a particular event, or combination of events.
 - These are called **indefinite** or **indeterminate** loops



Counted Loops

- A counted loop is simplest kind of loop
 - The loop bounds is controlled by a counter
 - Use it to for "perform this action 10 times"
- Java's **for** loop **specializes** in this

```
for (int count = 0; count < 10; count++)  
{  
    System.out.println("count="+count);  
}
```

The for Loop Syntax

- The **for** loop has both a header and body (like **if**)

```
1 Keyword  for ( 2 Initializer expression; 3 Boolean test expression; 4 Update expression )
           {
             5 Loop Body
           }
```

- Header has 3 expressions: **initialize**, **test**, and **update**

Initialization Expression

- The initialization expression
 - Executed once, before the loop is entered
 - Always executed, regardless of the test
 - Usually used to declare your counter variable

```
for (int counter = 0; ... ; ... )
{
    // Can use counter here
}
// Cannot use counter here
```

The Test Expression

- Works just like the test in an **if** statement
 - If true, loop body is entered. If false, it is not.
 - Always evaluated at least once
 - Also ends with a semicolon (like initialization)

```
for (int counter = 0; counter < 10; ... )
{
    // Can use counter here
}
// Cannot use counter here
```

The Update Expression

- Executed **after** the **body** of the loop is finished
 - Always executed one fewer times than test
 - Usually used to update your counter
 - Not** followed by a semicolon

```
for (int counter = 0; counter < 10; counter++)  
{  
    // Can use counter here  
}
```

// Cannot use counter here

Express Yourself

- Exercise 7:** import **EX_13_7** and **EX_13_7Test**
Open EX_13_7 and complete the code for the **doubleChar()** method. The method takes one **String** parameter and returns a new **String** with every character in the first doubled.
- Questions you should ask?
 - Can we figure out how many times we need to loop?
 - If so, then we can use a counter-controlled for loop
 - What output is needed? What do we do inside the loop?

Testing your Code

- Open EX_13_7Test and use Quick Fix to configure

Exercise 7: Run tests and snap a pic of all tests passing

Right-click here

Click Fix Project Setup

Loops and Strings

- Some tips to help you process **Strings** with loops
 - Use the **length()** method to visit every character
 - Use the **charAt()** or **substring()** method to extract
 - The **indexOf()** method is also useful

```
int len = str.length();
for (int i = 0; i < len; i++)
{
    char ch = str.charAt(i);
    String strCh = str.substring(i, i+1);
}
```

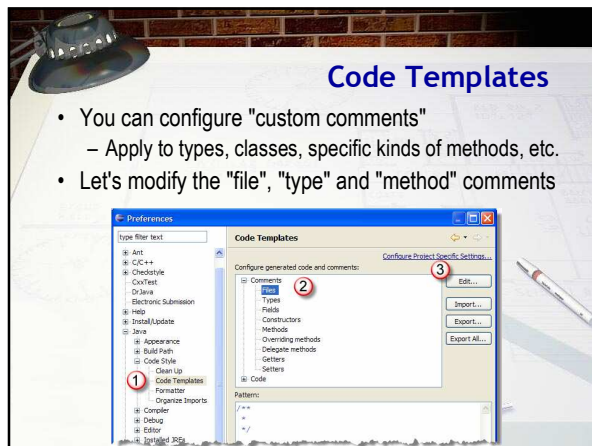
Another Exercise

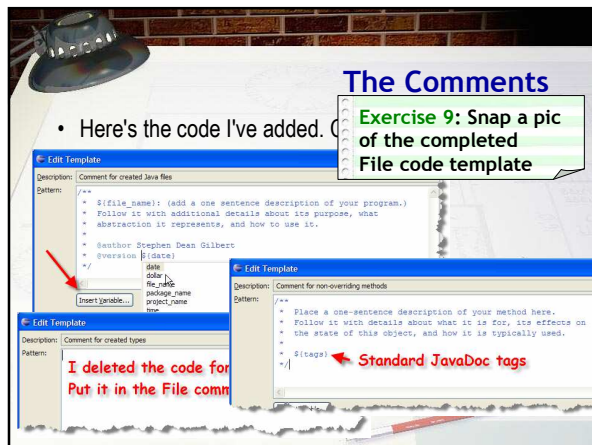
- Exercise 8:** import **EX_13_8** and **EX_13_8Test**
Open **EX_13_8** and complete the code for the **countHi()** method. The method takes one **String** parameter and returns the number of times that the **String** "hi" appears inside the first **String**.
 - Use **substring(i, i+2)** and the **equals()** method
 - Don't want to go through entire string
 - Can end up looking "past end" which causes a crash
 - No need for special case
- Right-click **EX_13_8Test** and run tests. Shoot a pic.

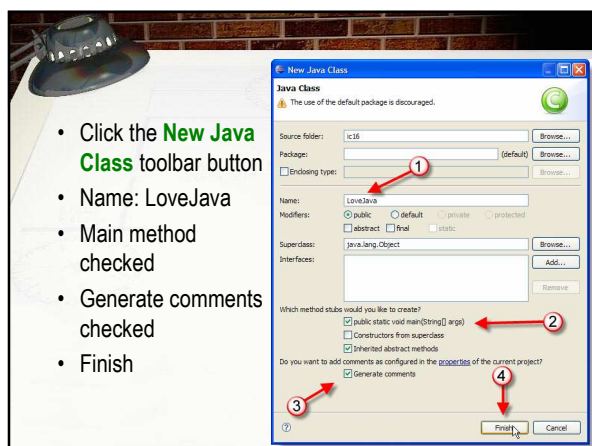
Writing New Code

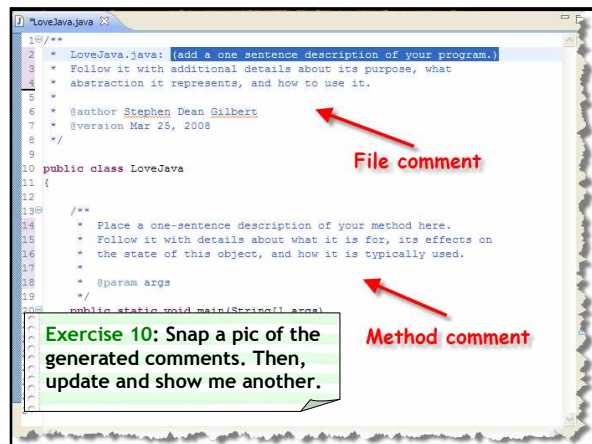
- Real power of Eclipse comes with code from scratch
 - We'll see how to create several kinds of programs
- Start with a simple console program that uses a counted loop to produce this output when run:

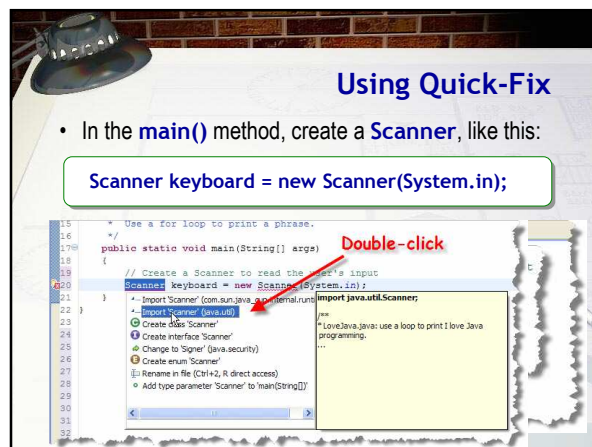
```
Enter a positive integer: 3
1 I love Java programming!!
2 I love Java programming!!
3 I love Java programming!!
Printed this message 3 times.
The sum of the numbers from 1 to 3 is 6.
```

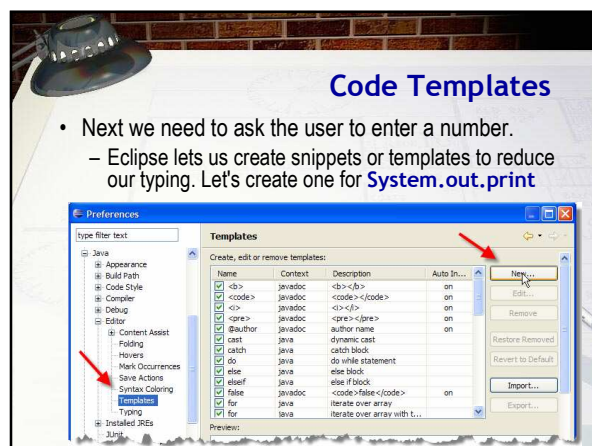


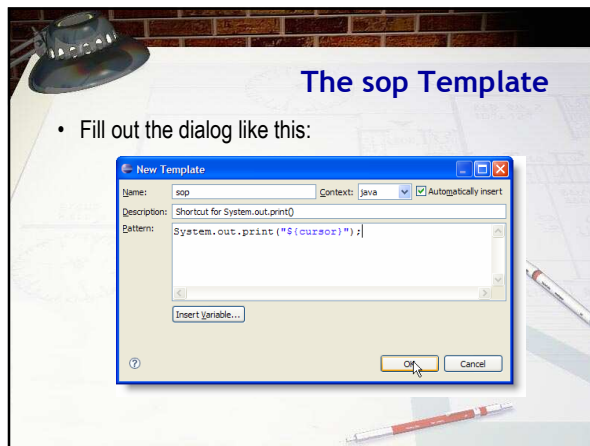


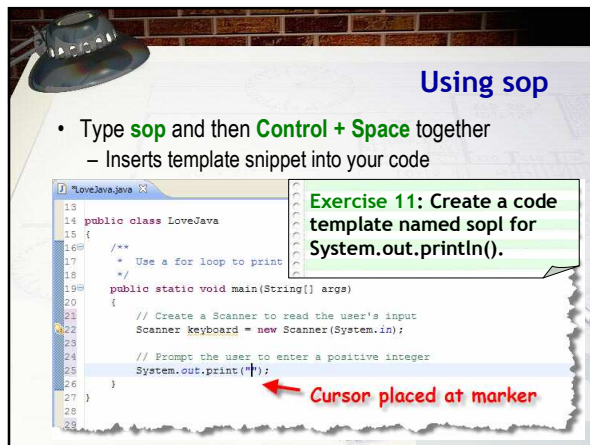


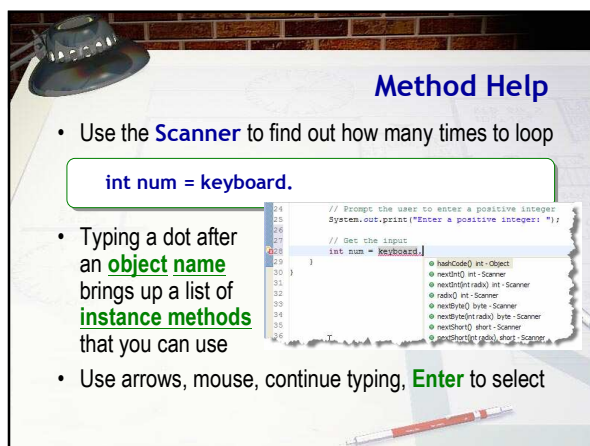












Exercise 12: Fix the error below by creating a variable `sum`, and adding up the numbers in the loop. Snap a pic.

Finishing Up

- Now, add a **for** loop that goes from 1 to `n`
 - Print your message inside the loop body
 - After the loop print the two "summary" lines

```
// Loop through num times;
for (int i = 1; i <= num; i++)
{
    System.out.println(i + " I love Java programming!!");
}

System.out.println("Printed this message " + num + " times.");
System.out.print("The sum of the numbers from 1 to " + num);
System.out.println(" is " + sum);
```

Diagram illustrating the loop structure and the need to add a `sum` variable. Red arrows point to the loop body and the final print statement.

Counting Down

- The **for** loop is a general purpose loop
 - Don't have to use a simple, incrementing counter
- You don't have to count up:

```
for (int i = 10; i > 0; i--) { ... }
```
- Exercise 13:** Open the file **StringReverse.java** and use a **for** loop to reverse the characters in the input **String**. Run and shoot a screenshot.
 - Remember **Strings** are numbered starting with 0

Counting by Steps

- You don't have to count by one:

```
for (int i = 0; i < 50; i += 5) { ... }
```

 - Works like BASIC's **STEP** command
 - Not restricted to a constant step, like BASIC

```
for (int i = 2; i < 10000 ; i * = 2 )
{
    System.out.println(i);
}
```
